

Task 1 - Understanding the Problem with Large Images

This task demonstrates how a single-stage Docker build can create unnecessarily large images because it includes the runtime, build tools, dependencies, and source files in the same layer structure.

```
-> Create project folder: mkdir large-image-demo
cd large-image-demo
-> Create Node app
  npm init -y
-> Build Docker image: docker build -t large-node-app .
-> Check image size :docker images
```

Task 2 - Multi-Stage Build

- Multi-stage builds separate the build environment from the runtime environment. Dependencies and build tools are used only in the builder stage, while the final image contains only the minimal runtime components required to run the application.

-> Example Multi-stage Dockerfile

```
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app .
EXPOSE 3000
CMD ["node","app.js"]
```

```
- Build image: docker build -t multi-stage-node .
- Compare sizes: docker images
```

Task 3 - Push Image to Docker Hub

This task demonstrates how to publish Docker images to Docker Hub. This allows images to be stored remotely and shared with other systems or team members.

- Login to Docker Hub: `docker login`
- Tag image: `docker tag multi-stage-node yourusername/node-demo:v1`
- Push image: `docker push yourusername/node-demo:v1`
- Verify by pulling: `docker pull yourusername/node-demo:v1`

Task 4 – Docker Hub Repository

After pushing the image, you can manage it from Docker Hub. You can add descriptions, view image layers, and manage version tags.

- Pull latest version: `docker pull yourusername/node-demo`
- Pull specific version: `docker pull yourusername/node-demo:v1`

Task 5 – Docker Image Best Practices

Best practices improve security, performance, and maintainability of Docker images. Key practices include using minimal base images, avoiding root users, combining commands to reduce layers, and using fixed image tags instead of 'latest'.

Optimized Dockerfile example

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
COPY . .
RUN addgroup appgroup && adduser -S appuser -G appgroup
USER appuser
EXPOSE 3000
CMD ["node", "app.js"]
```

Image Size Comparison

Image Type	Approx Size
Single Stage Image	≈ 900MB
Multi-Stage Image	≈ 120MB
Optimized Image	≈ 80MB